

# Spring AOP 原理和实现—复习要点

## 1 AOP 思想概述

### 1.1 为什么需要 AOP?

[p.6–9]

> **OOP 的局限性** (Limitations of OOP) [p.6]:

- 当需要在分散的、不具有继承层次的对象中引入公共行为时，OOP 无法避免代码重复
- OOP 允许定义从上到下的关系（纵向继承），但不适合定义从左到右的关系（横向关注）
- 大型系统中的日志记录、性能监控、安全认证、事务控制等功能，代码平均分散在所有对象层次中，与核心业务功能关系不大

> **传统方案的痛点** [p.8–9]:

- 纵向继承试图解决耦合和重复问题，但不够灵活 [p.8]
- **代码重复**：每个业务方法都需编写相同的日志/权限代码 [p.9]
- **耦合度高**：横切逻辑与核心业务紧密耦合，修改影响面大 [p.9]
- **难以维护**：跨模块实现一致更新困难 [p.9]

> **AOP 的解决思路** [p.9]：通过切面 (Aspect) 将横切逻辑抽取出来，统一织入，保持业务代码简洁，方便横切逻辑的修改和复用

### 1.2 什么是 AOP?

[p.10]

> **AOP** (Aspect Oriented Programming)：面向切面编程，是对 OOP 的一种完善和补充 [p.10]

> 1990 年由 PARC (Xerox Palo Alto Research Center) 研究人员提出 [p.10]

> AOP 采取**横向抽取机制**，取代传统的纵向继承体系解决重复性代码问题 [p.10]

> 典型应用场景：日志记录、安全检查、事务管理、性能监视、数据缓存 [p.10]

### 1.3 AOP 核心概念术语

[p.11–13]

> **关注点** (Concern) [p.11]：一个特定的问题、概念或应用程序的某部分，即程序必须达到的一个目标

- **核心关注点** (Core Concerns)：完成核心业务逻辑的关注点
- **横切关注点** (Cross-cutting Concerns)：代码散落在很多个类或方法之中的关注点

> **切面** (Aspect) [p.11]：对横切关注点的模块化，将散落在各处的横切代码归整在一起。一般包含切点定义和一个或多个**通知方法**

> **连接点** (Join Point) [p.13]：程序执行过程中的某个中断点（方法调用、字段访问、异常抛出），Spring AOP 仅支持**方法调用**作为连接点。是能插入切面的候选点集合

> **切点** (Pointcut) [p.13]：从候选连接点中选择的确定的插入切面的连接点，Spring AOP 中用**表达式**选择

> **通知/建议** (Advice) [p.13]：在切点上运行切面的方式，分为前置、后置、环绕、异常、最终五种

> **织入** (Weaving) [p.13]：将切面应用到原始/目标对象的方式，分为动态织入和静态织入

- > **动态横切 (Dynamic Crosscutting)** [p.12]: 运行时通过切入点匹配连接点并执行通知来增强代码。核心技术: 连接点、切入点、通知、切面
- > **静态横切 (Static Crosscutting)** [p.12]: 编译时/编译后不改变原始代码, 向类中引入新成员 (字段、方法) 或修改继承关系。核心技术: 引入/混入 (Introduction/Mixin)

2 AOP 核心概念速查表

| 术语  | 英文           | 说明                                |
|-----|--------------|-----------------------------------|
| 关注点 | Concern      | 程序必须达到的某个目标, 分核心关注点和横切关注点 [p.11]  |
| 切面  | Aspect       | 横切关注点的模块化封装, 含切点 + 通知 [p.11]      |
| 连接点 | JoinPoint    | 程序执行的候选中断点, Spring 仅支持方法调用 [p.13] |
| 切点  | Pointcut     | 从连接点中选出的确定位置, 用表达式定义 [p.13]       |
| 通知  | Advice       | 在切点上执行的动作, 五种类型 [p.13]            |
| 织入  | Weaving      | 将切面应用到目标对象的过程 [p.13]              |
| 引入  | Introduction | 静态横切: 向类中引入新方法/字段 [p.12]          |

3 AOP 的实现方式

3.1 三种实现原理 [p.16]

- > **编译期 (语言扩展) (Compile-time)** [p.16]: 由编译器将切面增加到字节码, 需定义新关键字和扩展编译器。代表: AspectJ 框架
- > **类加载器 (Class Loader)** [p.16]: 目标类装载到 JVM 时, 通过特殊类加载器对字节码重新”增强”
- > **运行时 (动态代理) (Runtime / Dynamic Proxy)** [p.16]: 目标对象和切面都是普通 Java 类, 通过 JVM 动态代理或第三方库实现运行期动态织入。代表: JDK 动态代理、CGLIB

3.2 JDK 动态代理 [p.17–21]

- > 基于 Java 反射机制, 要求目标类必须实现**接口** [p.17]
- > 通过 `java.lang.reflect.Proxy` 和 `InvocationHandler` 在运行时动态生成代理类 [p.18–20]
- > 代理类实现了目标接口, 方法调用时通过 `InvocationHandler` 转发到目标对象 [p.21]
- > **局限**: 只能代理实现了接口的类 [p.30]

3.3 AspectJ 框架 [p.22–27]

- > 基于 Java 语言的**最完整 AOP 框架**, 功能最全面 [p.22]
- > 通过专门的编译器 (`ajc`) 或类加载器在编译期/编译后/加载时织入切面 [p.22]
- > 支持强大的**静态织入**和**动态织入**能力 [p.22]
- > 静态横切只能用 **AspectJ** 实现 [p.22]

- > 提供 **Introduction / Mixin** 功能：向已有类引入新接口和方法 [p.24-25]
- > **缺点**：需引入第三方库，学习成本较高 [p.22]

## 4 Spring AOP

### 4.1 Spring AOP 简介

[p.29]

- > Spring 支持多种 AOP 实现：JDK 代理、CGLIB 字节码代理、AspectJ [p.29]
- > 实际业务应用中只涉及**动态横切**，不推荐基于 AspectJ 的静态横切 [p.29]
- > 从 Spring 3.x 起，建议使用 **AspectJ 注解方式**实现 AOP [p.29]
- > 方式有两种：XML 配置和注解方式 [p.29]

### 4.2 Spring AOP 代理模式

[p.30]

- > **JDK 动态代理** [p.30]:
  - 目标 Bean 实现一个或多个接口时，Spring 默认使用 JDK 动态代理
  - 基于接口的代理，代理对象和目标对象实现相同接口
- > **CGLIB 字节码代理** [p.30]:
  - 目标类没有实现接口，或设置 `proxyTargetClass=true` 时使用
  - 通过创建目标类的子类来实现代理
  - **注意**：final 类和 final/private 方法不能被 CGLIB 代理
- > Spring 7 引入 `@Proxyable` 注解设置全局代理策略 [p.30]

### 4.3 JDK 代理 vs CGLIB 代理对比

| 特性        | JDK 动态代理 | CGLIB 代理       |
|-----------|----------|----------------|
| 代理方式      | 基于接口     | 基于子类继承         |
| 要求        | 必须有接口    | 不能是 final 类/方法 |
| 性能        | 反射调用，略慢  | 字节码生成，较快       |
| 默认使用      | 有接口时默认   | 无接口或强制指定时      |
| Spring 推荐 | 接口代理优先   | 补充方案           |

### 4.4 切点表达式语法

[p.32]

- > **execution**：匹配方法执行，最常用的设计器 [p.32]
  - `execution(* com.example.service.*(..))` 匹配指定包下所有方法
- > **within**：限制切面只在某个包或类内生效，如 `within(com.example.service...)` [p.32]
- > **args**：按参数类型匹配，如 `args(java.lang.String, ..)` [p.32]
- > **@annotation / @within / @target**：按注解匹配 [p.32]
- > 切点表达式支持逻辑运算：**&&**（与）、**||**（或）、**!**（非） [p.32]
- > 可用 `@Pointcut` 定义切点并复用 [p.32]

## 4.5 五种通知类型

[p.33]

| 注解                           | 类型   | 执行时机          | 适用场景         |
|------------------------------|------|---------------|--------------|
| <code>@Before</code>         | 前置通知 | 目标方法执行前       | 参数校验、前置日志    |
| <code>@AfterReturning</code> | 后置返回 | 方法正常返回后       | 捕获返回值（不可修改）  |
| <code>@AfterThrowing</code>  | 异常通知 | 方法抛出异常后       | 异常日志、告警      |
| <code>@After</code>          | 最终通知 | 方法结束时（无论是否异常） | 资源清理         |
| <code>@Around</code>         | 环绕通知 | 方法调用前后        | 事务管理、缓存、性能监控 |

- > **通知优先级** [p.33]: 多个切面作用于同一连接点时, 通过 `@Order` 注解 (数值越小优先级越高) 或实现 `Ordered` 接口控制顺序
- > **最佳实践**: 能用其他通知解决时, 不要滥用环绕通知 [p.33]

## 5 Spring AOP 实际项目使用

### 5.1 自定义切面注解方式

[p.35–38]

- > 直接使用切面自动装配时, 控制切面使用范围不够精确 [p.35]
- > **推荐做法** [p.36]: 将切面定义成自定义注解, 精确控制切入位置
- > **范例**: 自定义 `@PerformanceMonitor` 注解实现性能监控切面 [p.37–38]
  - 定义注解 `@interface PerformanceMonitor`
  - 编写切面类, 用 `@Around("@annotation(xxx)")` 绑定
  - 在需要监控的方法上添加注解即可

### 5.2 自调用导致 AOP 失效

[p.39]

- > **问题**: 同一类中方法 A 调用方法 B, B 上的 AOP 注解不生效
- > **原因**: 自调用不经过 Spring 代理对象, 而是直接调用 `this.methodB()`
- > **解决方案**:
  - 通过 `AopContext.currentProxy()` 获取当前代理对象再调用
  - 或将被调用的方法抽取到另一个 Bean 中
  - 或在配置中开启 `exposeProxy=true`

### 5.3 常用 AOP 组件/模块

[p.40]

- > **事务管理** (`@Transactional`): 基于环绕通知, 方法调用前开启事务, 正常返回后提交, 抛异常时回滚 [p.40]
- > **缓存** (`@Cacheable`): 环绕通知检查缓存命中, 跳过方法执行返回缓存结果, 否则执行方法并写入缓存 [p.40]
- > **异步执行** (`@Async`): 代理中将方法执行提交到线程池, 实现异步调用 [p.40]
- > **重试与限流** (`@Retryable` / `@ConcurrencyLimit`): 拦截方法执行重试逻辑或限制并发调用数 [p.40]

## 6 IoC 与 AOP 原理总结

### 6.1 架构全景图

[p.42]

- > 整个 Spring 架构基于 **IoC/Bean 容器**, 在其上通过 AOP 实现横切功能 [p.42]

- > 表示层 (Servlet/JSP) → 业务逻辑层 (Service/Business Bean) → 模型层 (DAO) 的纵向调用 + AOP 横向织入 [p.42]
- > AOP 横切三大类：权限/安全管理切面、日志管理切面、事务管理切面 [p.42]

## 7 本章小结与考点梳理

### 7.1 核心知识体系

[p.1–42]

- > **AOP 思想** [p.6–10]：为什么需要 AOP（OOP 局限、横切关注点分散、代码重复）、AOP 定义（横向抽取机制）、典型场景
- > **AOP 核心概念** [p.11–13]：关注点 (Concern)、切面 (Aspect)、连接点 (JoinPoint)、切点 (Pointcut)、通知 (Advice)、织入 (Weaving)、动态横切 vs 静态横切
- > **AOP 实现方式** [p.16–27]：编译期 (AspectJ)、类加载器、运行时 (JDK 动态代理 / CGLIB)
- > **Spring AOP** [p.29–33]：JDK 代理 vs CGLIB 代理、切点表达式语法、五种通知类型 (@Before/@AfterReturning/@AfterThrowing/@Around/@Order 优先级)
- > **实战要点** [p.35–40]：自定义切面注解模式、自调用 AOP 失效问题及解决、常用 AOP 组件 (@Transactional/@Cacheable)

### 7.2 考试重点提示

高频考点：(1) AOP 七大核心概念中英文名称和含义 [p.11–13]；(2) OOP 的局限性与 AOP 的解决思路 [p.6, p.9]；(3) AOP 三种实现原理及代表技术 [p.16]；(4) JDK 动态代理 vs CGLIB 代理的对比 [p.30]；(5) 五种通知类型的注解、执行时机、适用场景 [p.33]；(6) 切点表达式语法（execution/within/args 及逻辑运算）[p.32]；(7) 自调用导致 AOP 失效的原因和解决方案 [p.39]；(8) Spring 中 AOP 实现的常用组件 [p.40]